Warehouse Management with Reinforcement Learning

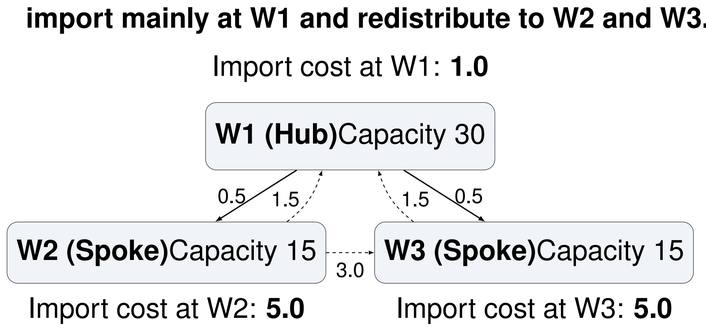
Elias Al Bouzidi, Massine El Khader, Idriss Benkirane

CentraleSupélec — université PARIS-SACLAY

1. Context & Operational Problem

- We study a **three-warehouse inventory control problem** in a hub-and-spoke logistics network.
- **W1** is the central hub, while **W2** and **W3** are spoke warehouses with lower capacity and more expensive direct imports.
- Each month, the controller decides how much to import and whether to transfer stock before stochastic demand is realized.
- The operational trade-off is between service quality, holding cost, transfer cost, and infeasible actions.
- Because replenishment is cheapest at the hub, the intended structure is:

import mainly at W1 and redistribute to W2 and W3.



Hub-and-spoke warehouse network with cheaper replenishment at the hub and asymmetric transfer costs.

2. Environment Design

Warehouses and costs

Warehouse	Role	Cap.	Import	Hold.
W1	Hub	30	1.0	0.3
W2	Spoke	15	5.0	1.0
W3	Spoke	15	5.0	1.0

Transfer	Cost
W1 → W2 / W3	0.5
W2 / W3 → W1	1.5
W2 ↔ W3	3.0

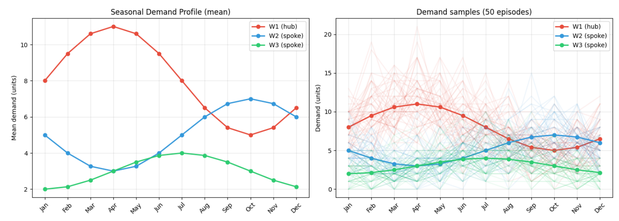
State

$$s = (\text{stock}_1, \text{stock}_2, \text{stock}_3, \text{month})$$

Demand

$$D_i(m) \sim \text{Poisson}(\mu_i(m))$$

$$\mu_i(m) = \max\left(1,\ \text{base}_i + \text{amplitude}_i \sin\left(\frac{2\pi(m - \text{phase}_i)}{12}\right)\right)$$



Seasonal demand profile and sampled monthly demand realizations.

Action space

- **34 discrete actions** combining imports and optional transfers.

Action type	Count
No-op	1
Single imports	9
Transfer-only	12
Bulk imports	2
Hub import + transfer	6
Hub-specific large actions	4

3. Decision Process & Reward

At each month, the controller:

1. applies imports,
2. applies transfers,
3. samples demand,
4. fulfills demand,
5. updates inventory,
6. computes the reward.

Reward

$$R_t = -10\sum_i \text{unmet}_i - \sum_i c_i^{\text{import}}\text{import}_i - \sum_{i,j} c_{ij}^{\text{transfer}}\text{transfer}_{ij} - \sum_i c_i^{\text{hold}}\text{stock}'_i - \text{infeasible units}$$

- Unmet demand is heavily penalized to prioritize service level.
- Imports and transfers contribute directly to operational cost.
- Holding cost discourages unnecessary overstocking.
- Infeasible actions are clamped and penalized.

Episode setup

- Horizon: **12 months**, Initial stock: **(15, 7, 7)** ,Discount factor: $\gamma = 0.95$.

4. Methods Compared

We compare **five discrete controllers** under the same 12-step seasonal environment.

- **Random**: uniform action selection over the 34 actions.
- **Heuristic**: hand-crafted hub-first controller.
- **Value Iteration**: finite-horizon planning,

$$V_t(s) = \max_a \mathbb{E}\big[R(s,a,D) + \gamma V_{t+1}(s')\big],$$

  with stock coarsening step **4**.
- **Q-Learning** and **SARSA**: tabular TD control with stock coarsening step **2**, $\alpha = 0.1$, $\gamma = 0.95$, and ε-greedy exploration from **1.0** to **0.05**.
- **DQN**: neural approximation $Q(s,a) \approx Q_\theta(s,a)$ with 6 inputs, two hidden layers of size 64, and 34 outputs.

Method	Type	State rep.	Main idea
Random	baseline	exact	no learning
Heuristic	rules	exact	hub-first control
Value Iteration	planning	step-4	finite-horizon DP
Q-Lear./ SARSA	tabular RL	step-2	TD updates
DQN	deep RL	normalized	function approx.

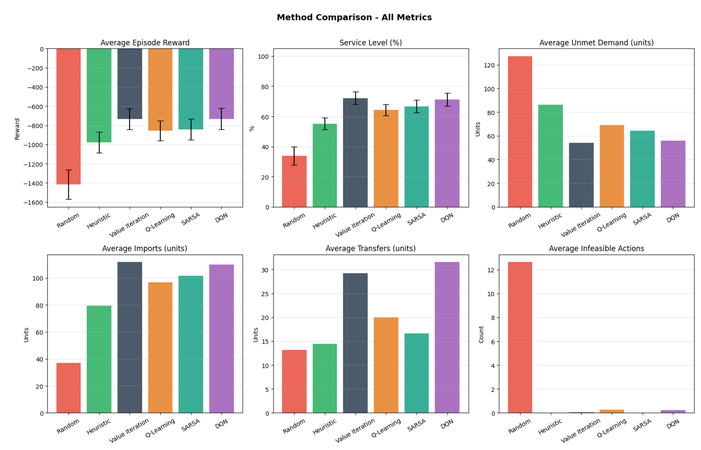
Protocol

- Value Iteration is solved by backward induction on the coarsened state space.
- Q-Learning, SARSA, and DQN are trained for **50,000 episodes**.
- Final policies are evaluated over **200 episodes**.
- Main metrics: reward, unmet demand, service level, imports, transfers, and infeasible actions.

Goal. A good policy should both improve reward and recover the intended rule:

import at W1, then redistribute to W2 and W3.

5. Discrete Results

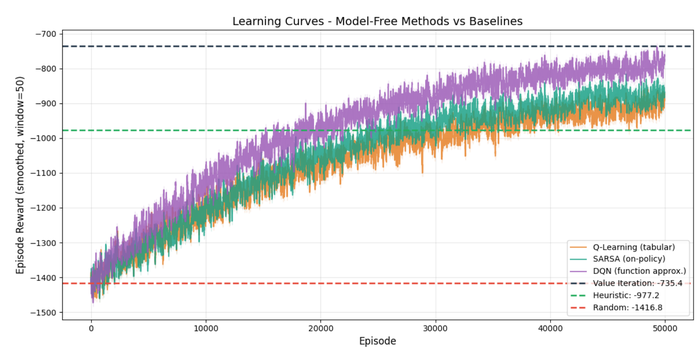


Discrete-method comparison on reward, service level, unmet demand, imports, transfers, and infeasible actions.

Method	Avg reward	Service (%)	Unmet
Random	-1416.75	34.0	127.25
Heuristic	-977.21	55.2	86.28
Value Iteration	-735.44	**72.1**	**54.04**
SARSA	-841.07	66.7	64.34
DQN	**-732.65**	71.2	55.85

- **DQN** gives the best average reward: **-732.65**.
- **Value Iteration** gives the best service level: **72.1%**.
- **SARSA** improves over tabular Q-Learning on reward, unmet demand, and feasibility.
- All informed methods drive infeasible actions close to zero, unlike the random baseline.

6. Hub Strategy & Takeaway



Learning curves for Q-Learning, SARSA, and DQN against the main baselines.

Warehouse-level imports and transfers show whether a policy learned the hub-and-spoke structure.

7. Continuous Extension

We extend to a **continuous-action** setting where the policy directly outputs shipment magnitudes between warehouses, instead of choosing from a fixed discrete list.

- For N **warehouses**, the action is a transfer matrix in $\mathbb{R}^{N^2}$. E.g. small continuous experiment, $N = 4$, so the policy controls **16** continuous values.
- Raw actions are produced in $[-1,1]^{N^2}$ and mapped to shipment quantities with

$$T = \text{clip}(a, 0, 1) \cdot I_{\max},$$

  then projected to satisfy feasibility constraints.
- This removes hand-crafted discrete combinations and lets the controller choose **how much** to send on each link.
- The cost matrix follows a hub-and-spoke topology : 1 hub, 3 spokes. Hub routes are cheapest.
- The gain is flexibility; the cost is a harder optimisation problem with a high-dimensional action space.

Reward

$$R_t = -C_{\text{transport}} - \lambda_d \sum_i U_i$$

where $C_{\text{transport}} = \sum_{ij} c_{ij} T_{ij}$, $U_i = \max(0, D_i - I_i^{\text{post}})$ is unmet demand.

8. PPO for Continuous Control

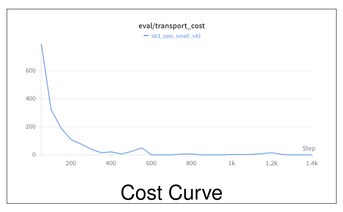
PPO handles Gaussian continuous policies and keeps updates stable via the clipped surrogate:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\Big(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big]$$

Settings: 14 parallel environments, 50 steps per rollout, 10 epochs, batch 256, lr 3×10^{-4}, γ=0.99, λ_{GAE}=0.95, network $[128, 128]$.

Parameter	Value	Role
`clip_range`	$0.2 \to 0.0$	Large updates early (explore), small updates late (exploit)
`log_std_init`	-1.0	Initial std $\approx$ 0.37; default 1.0 causes excessive noise

Reward Curve



Cost Curve

9. Continuous Results & Conclusion

Discrete Environment.

- We designed an inventory-control environment where the cost structure naturally favors **hub-first replenishment**.
- Reinforcement learning improves performance substantially over random behavior.
- The strongest methods are **Value Iteration** and **DQN**: one is best on service, the other on reward.
- The best learned policies improve reward and recover the intended operational behavior:

import at W1, then redistribute to W2 and W3.

Continuous Environment.

- In the continuous proposal, PPO improves strongly during training: reward rises from roughly **-16,000** to **-7,000**, while transport cost drops from about **800** to **10**.
- Final demand satisfaction reaches about **78%**, comparable to the greedy baseline but with much lower transport cost.
- The remaining gap is explained by stochastic demand, delayed credit assignment, and the difficulty of learning in a **16-dimensional** continuous action space.

Takeaway.
This project shows that RL can learn efficient and interpretable warehouse policies when the environment and action design are aligned with realistic operational costs.

Limitation & future work. Stochastic demand makes credit assignment hard: a good shipping decision may still yield unmet demand . Future work could incorporate demand forecasting or a GNN policy with explicit hub-spoke structure to close the remaining gap.